

1. COS 226, f25, precept 1. Tìm bug

Hãy xem cài đặt không chính xác sau đây của hàm `union()` trong cấu trúc dữ liệu quick-find. Lưu ý rằng mảng `leader[]` có độ dài n được khởi tạo với `leader[i] = i` cho mọi i , và hàm `find(i)` trả về giá trị `leader[i]`.

```
1 public void union(int p, int q) {
2     for (int i = 0; i < leader.length; i++)
3         if (leader[i] == leader[p])
4             leader[i] = leader[q];
5 }
```

Hãy một giá trị cho số lượng phần tử n , một chuỗi các phép toán `union()`, một số nguyên $0 \leq i < n$ và một số nguyên $0 \leq j < n$, sao cho các phần tử i và j thuộc cùng một tập hợp nhưng `find(i)` và `find(j)` trả về các giá trị khác nhau. Nói cách khác là tìm 1 testcase mà cài đặt trên chạy sai.

Chọn $n = 3$. Chuỗi phép toán:

- `union(0, 1)`
- `union(0, 2)`

Trạng thái ban đầu: `leader = [0, 1, 2]`

Chạy lệnh 1 - `union(0, 1)`:

- o Mảng được cập nhật đúng. Trạng thái sau lệnh 1: `leader = [1, 1, 2]`. Lúc này phần tử 0 và 1 đang cùng một nhóm (có chung leader là 1).

Chạy lệnh 2 - `union(0, 2)`:

- o $p = 0$ và $q = 2$.
- o Khi $i = 0$: `leader[0] == leader[0]` đúng. Lệnh gán chạy: `leader[0] = leader[2]` (tức là `leader[0] = 2`).

Mảng tức thời biến thành: `[2, 1, 2]`.

Khi $i = 1$: kiểm tra `leader[1] == leader[0]`. Máy tính lấy `leader[1]` (đang là 1) so sánh với `leader[0]` (bị đổi thành 2).

Vì sai nên lệnh gán không được thực thi.

- o Khi $i = 2$: `leader[2] == leader[0]` ($2 == 2$) đúng. Gán lại bằng 2.

Vậy mảng cuối cùng là `[2, 1, 2]`. Ta chọn $i = 0$ và $j = 1$. Dù về chúng đã được gộp vào cùng một tập hợp từ lệnh đầu tiên, nhưng hàm `find(0)` lại trả về 2, trong khi hàm `find(1)` lại trả về 1

2. COS226, Midterm f22

Giả sử ta khởi tạo một cấu trúc dữ liệu union-find với n phần tử. Sau đó, ta thực hiện chuỗi các thao tác `union()` sau: `union(0, 1)`, `union(0, 2)`, `union(0, 3)`, ..., `union(0, n-1)`.

Trả lời các câu hỏi sau:

(a) Cấu trúc dữ liệu thu được chứa tổng cộng bao nhiêu thành phần liên thông (tức là các tập hợp rời nhau)?

Tất cả mọi phần tử đều đã được móc nối trực tiếp với số 0. Không có phần tử nào bị bỏ rơi lẻ loi bên ngoài, nên toàn bộ mảng hợp thành một khối duy nhất.

(b) Giả sử cấu trúc dữ liệu được cài đặt theo phương pháp quick-find. Có bao nhiêu lần cập nhật mảng được thực hiện bởi các thao tác `union()` này (biểu diễn dưới dạng hàm của n theo ký hiệu xấp xỉ $\sim$)? (Lưu ý: cài đặt quick-find cho `union(p, q)` của chúng ta không bao giờ thay đổi giá trị `leader[q]`.)

Chuỗi lệnh của chúng ta có dạng `union(0, i)` với i chạy từ 1 đến $n-1$

Lệnh 1 - `union(0, 1)`:

- Nhóm của 0 ban đầu chỉ có 1 mình số 0.
- Ta phải đổi `leader[0]` sang giống `leader[1]`.
- Số lượng mảng bị cập nhật: **1** thao tác. (Nhóm của 0 giờ có 2 thành viên là $\{0, 1\}$).

Lệnh 2 - `union(0, 2)`:

- Nhóm của 0 hiện tại đang có 2 thành viên (là 0 và 1).
- Ta phải đổi cả `leader[0]` và `leader[1]` sang giống `leader[2]`.
- Số lượng mảng bị cập nhật: **2** thao tác. (Nhóm của 0 phình to thành $\{0, 1, 2\}$).

Lệnh 3 - `union(0, 3)`:

- Nhóm của 0 hiện tại đã có 3 thành viên (0, 1, 2).
- Phải cập nhật mảng cho cả 3 phần tử này sang giống `leader[3]`.
- Số lượng mảng bị cập nhật: **3** thao tác.

$$\text{Tổng} = 1 + 2 + 3 + \dots + (n-1) = (n^2)/2$$

(c) Giả sử cấu trúc dữ liệu được cài đặt theo phương pháp quick-union, và ta gọi `find(0)` sau chuỗi thao tác nêu trên. Thao tác `find(0)` sẽ thực hiện bao nhiêu lần truy cập mảng (biểu diễn dưới dạng hàm của n theo ký hiệu Θ)? (Lưu ý: cài đặt quick-union cho `union(p, q)` của chúng ta không bao giờ thay đổi `parent[q]`.)

chuỗi lệnh `union(0, i)` :

Vị trí p luôn là 0

Vị trí q luôn là số i

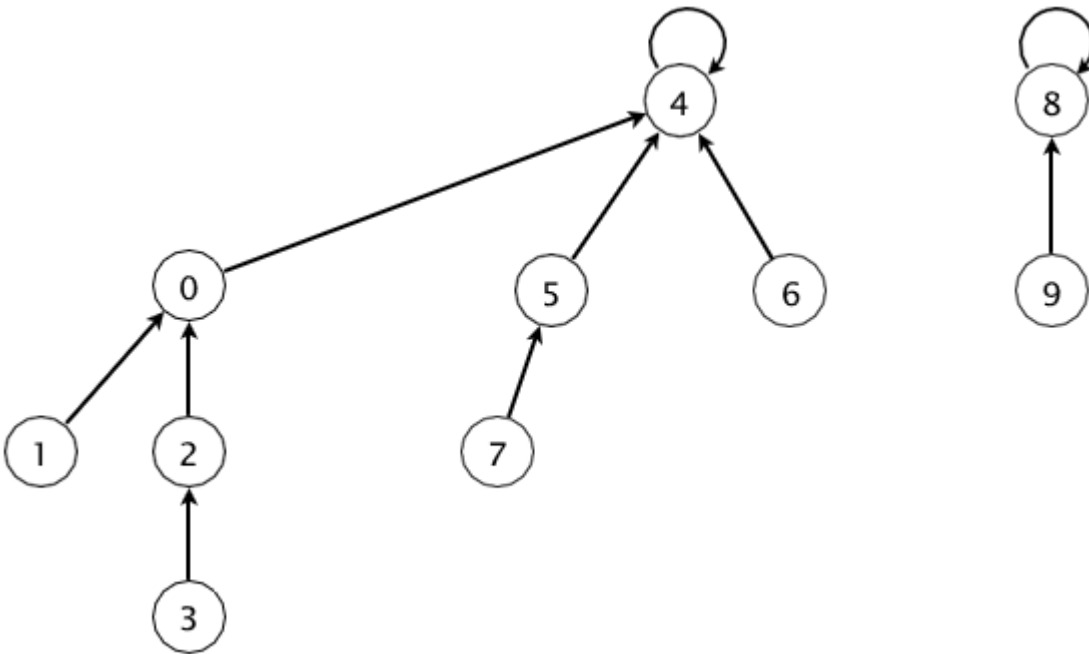
nó thực hiện một lệnh gán duy nhất: $\text{parent}[\text{rootP}] = \text{rootQ}$. Lệnh này mang toàn bộ cái cây của p bứng đi và móc vào làm cây con của gốc q

(d) Giả sử cấu trúc dữ liệu được cài đặt theo phương pháp weighted quick-union, và ta gọi $\text{find}(0)$ sau chuỗi thao tác nêu trên. Thao tác $\text{find}(0)$ sẽ thực hiện bao nhiêu lần truy cập mảng (biểu diễn dưới dạng hàm của n theo ký hiệu Θ)? (Lưu ý: cài đặt weighted quick-union cho $\text{union}(p, q)$ của chúng ta sẽ thay đổi $\text{parent}[q]$ nếu hai cây có cùng kích thước.)

Sẽ tạo thành 1 cây có số 0 ở trung tâm và các số còn lại nối với 0

3. COS226, Midterm f25, 2a

Xét biểu diễn *parent-link* dưới đây của một cấu trúc dữ liệu *weighted quick union* (*link-by-size*):



Trong các cặp (p,q) dưới đây, những cặp nào có thể là tham số của lệnh $\text{union}(p, q)$ gần nhất? Giả thiết rằng p và q thuộc hai tập khác nhau trước khi lệnh $\text{union}(p, q)$ được chạy.

(0, 4) (2, 3) (4, 0) (4, 5)

(5, 0) (5, 7) (6, 4) (8, 9)

(4, 0), (4, 5), (5, 0), (6, 4), và (8, 9).

(0, 4) loại do 0 nối đến 4 mà 2 cái dài = nhau nên phải là $\text{union}(p, q)$ thì q là 0 và p là 4

(5, 7) và (2, 3) loại do nó ko nối đến 4 hoặc 8

chú ý: khi gọi $\text{union}(p, q)$ mà cây chứa p và cây chứa q có kích thước bằng nhau thì ta sẽ trở nút gốc cây chứa q tới gốc của cây chứa p .

4. COS226, Midterm s25, 2a

Xét biểu diễn *parent-link* dưới đây của một cấu trúc dữ liệu *weighted quick union* (*link-by-size*):

parent[]	0	0	0	0	4	4	4	5	?	8
	0	1	2	3	4	5	6	7	8	9

parent[8] có thể nhận những giá trị nào trong các giá trị 0, 1,..., 9

parent[9] = 8, tức là 9 trở vào 8

- 8 trở vào chính nó (parent[8] = 8)
- 8 trở vào 4 (parent[8] = 4)
- 8 trở vào 0 (đc vì size của 0 lớn hơn 2)
- 8 trở vào 5 (loại vì 5 trở đến 4 mà size của 4 là 2 khi chưa nối với 5 mà size của 5 lúc 8 trở đến là 4 nên ko 5 ko trở đến 4 đc)
- 8 trở đến 1, 2, 3, 6, 7, 9 (loại vì size của các số này là 1 nên 8 ko trở đến đc)

5. COS226, Midterm f24, 4

Dưới đây có thể là biểu diễn *parent-link* của một cấu trúc dữ liệu *weighted quick union* hay không?

0	1	2	3	4	5	6	7	8	9
5	6	0	4	0	5	0	5	6	7

- Nút 0 bao gồm {0, 2, 4, 6, 3, 1, 8}
- Vì parent[0] = 5 nên lúc 0 nối vào 5 thì nó phải đầy đủ rồi (do 0 ko còn là gốc nữa)
- Mà size của 0 là 7 mà size của 5 khi chưa đc 0 trở vào là 3 (nên ko thể là WUC đc)